

FRUTIFICAR DIGITAL

Documento de Contexto

Escopo · Arquitetura · Design System · Boas Práticas

Versão 1.0 — Maio de 2026

Studio Need · Cássio Bispo Ramos Pereira

Cliente: Douglas Vargas Garcia

Sumário

1. Visão geral do produto
2. Stack e arquitetura técnica
3. Modelagem de dados
4. Estrutura de pastas e código
5. Design system
6. Estrutura de rotas
7. Fluxos críticos
8. Boas práticas de desenvolvimento
9. Convenções de código
10. Checklist por sprint

1. Visão geral do produto

O **Frutificar Digital** é uma plataforma web educacional e de gestão voltada ao setor agrícola. Funciona como um ambiente de reprodução de vídeos (estilo Netflix) organizado em cursos e módulos, complementado por funcionalidades de suporte, agendamento, comunicação e gestão da propriedade rural.

1.1. Público-alvo

- Produtores rurais que buscam capacitação técnica em café e gestão da propriedade.
- Estudantes de agronomia e áreas correlatas.
- Profissionais que oferecem suporte técnico no campo.

1.2. Modelo comercial

Três planos de assinatura mensal recorrente:

Funcionalidade	Essencial	Premium	Gold
Curso principal de café	✓	✓	✓
Chat IA com suporte	✓	✓	✓
Diagnóstico por imagem (WhatsApp)	✓	✓	✓
Minicursos com instrutores convidados	—	✓	✓
Agendamento de visita técnica	—	✓	✓
Menu de gestão (downloads)	—	✓	✓
Solicitação de serviços	—	✓	✓
Podcasts exclusivos	—	✓	✓
Lives exclusivas	—	✓	✓
Agenda de dias de campo	—	—	✓
Tutoria e estágio supervisionado (info)	—	—	✓

1.3. Princípios do produto

- **Mobile-first responsivo** — a maior parte dos usuários acessa pelo celular, mas o sistema é web (não app nativo).

- **Foco em vídeo** — toda a experiência gira em torno do consumo de aulas e lives.
- **Simplicidade pro produtor rural** — fluxos diretos, poucos cliques, terminologia acessível.
- **Controle de acesso rigoroso por plano** — cada feature precisa validar o plano do usuário.

2. Stack e arquitetura técnica

2.1. Stack principal

Camada	Tecnologia	Versão
Frontend + Backend	Next.js (App Router)	14.x
Linguagem	TypeScript	5.x
Runtime	Node.js	20.x LTS
Banco de dados	PostgreSQL	16.x
ORM	Prisma	5.x
Autenticação	NextAuth.js (Auth.js v5)	5.x
Estilização	Tailwind CSS	3.x
Componentes UI	shadcn/ui	latest
Validação	Zod	3.x
Formulários	React Hook Form	7.x
Data fetching	TanStack Query (React Query)	5.x
Ícones	Lucide React	latest

2.2. Serviços externos

Serviço	Função
OpenAI API	Chat inteligente com streaming
YouTube Data API v3	Sincronização de status de lives e embed de vídeos
Gateway de pagamento (Asaas sugerido)	Processamento de assinaturas e webhooks
Resend ou SendGrid	E-mails transacionais
AWS S3 ou Cloudflare R2	Armazenamento de arquivos (PDFs, imagens, certificados)

2.3. Princípios de arquitetura

Server-first. Use Server Components por padrão. Só vire Client Component quando precisar de interatividade (estado, eventos, hooks). Isso reduz JS no cliente e melhora performance.

API próxima da UI. Use Server Actions e Route Handlers do Next.js. Evite criar um backend separado.

Type-safety end-to-end. Prisma gera tipos do banco, Zod valida entradas, TypeScript propaga tipos. Nunca use `any`.

Single source of truth. O banco define o schema, o Prisma gera os tipos, o resto do código consome esses tipos. Não duplique definições.

2.4. Camadas do sistema

```
UI (React Server + Client Components)
```

```
Server Actions / Route Handlers (API)
```

```
Services (lógica de negócio)
```

```
Repositories (acesso a dados)
```

```
Prisma Client → PostgreSQL
```

3. Modelagem de dados

3.1. Entidades principais

Usuários e acesso

- **User** — id, email, name, passwordHash, role (STUDENT, ADMIN, INSTRUCTOR), createdAt.
- **Plan** — id, name (ESSENCIAL, PREMIUM, GOLD), priceMonthly, features (JSON), maxProperties.
- **Subscription** — id, userId, planId, status (ACTIVE, PAST_DUE, CANCELED), currentPeriodEnd, gatewayCustomerId, gatewaySubscriptionId.
- **CourseAccess** — pivô planId × courseId.

Conteúdo educacional

- **Course** — id, title, slug, description, coverImage, type (PRINCIPAL, MINICOURSE), instructorId, published.
- **Module** — id, courseId, title, order.
- **Lesson** — id, moduleId, title, description, youtubeVideoId, durationSec, order.
- **Material** — id, lessonId, title, fileUrl, type (PDF, SPREADSHEET, DOC).
- **Enrollment** — id, userId, courseId, startedAt.
- **LessonProgress** — id, userId, lessonId, completed, completedAt.
- **Certificate** — id, userId, courseId, certificateUrl, issuedAt.

Comunicação

- **ChatSession** — id, userId, messages (JSON), lastMessageAt.
- **Live** — id, courseId, title, youtubeVideoId, scheduledAt, status (SCHEDULED, LIVE, ENDED), requiredPlan.
- **Podcast** — id, title, description, coverImage.
- **PodcastEpisode** — id, podcastId, title, embedUrl ou audioUrl, publishedAt.

Serviços e agendamentos (Premium e Gold)

- **TechnicalVisit** — id, userId, propertyId, reason, requestedDate, status (REQUESTED, CONFIRMED, COMPLETED, CANCELED), notes.
- **ServiceRequest** — id, userId, serviceType, description, status (OPEN, IN_ANALYSIS, IN_PROGRESS, COMPLETED, CANCELED), adminResponse.
- **DownloadableResource** — id, title, description, fileUrl, category, requiredPlan.
- **FieldDay** — id, title, location, date, instructor, description.

Gestão da propriedade rural

- `Property` — id, userId, name, totalAreaHa, location.
- `Plot` — id, propertyId, name, areaHa, status.
- `SoilAnalysis` — id, plotId, ph, nutrients (JSON), analyzedAt.
- `Activity` — id, plotId, type, description, performedAt.
- `Recommendation` — id, plotId, content, priority, generatedFromRuleId.
- `RecommendationRule` — id, condition (JSON), recommendationText, active.

3.2. Convenções de schema

- IDs sempre como `String @id @default(cuid())`.
- Timestamps automáticos: `createdAt DateTime @default(now())` e `updatedAt DateTime @updatedAt`.
- Soft delete opcional via campo `deletedAt DateTime?` em entidades críticas.
- Enums em SCREAMING_SNAKE_CASE no Prisma, convertidos no domínio.
- Relacionamentos sempre com `onDelete` explícito.

4. Estrutura de pastas e código

```
frutificar-digital/
├─ prisma/
│  ├─ schema.prisma
│  └─ migrations/
│     └─ seed.ts
├─ public/
│  ├─ logo.svg
│  └─ images/
├─ src/
│  ├─ app/
│  │  ├─ (public)/                # rotas públicas (landing, planos, login)
│  │  │  ├─ page.tsx
│  │  │  └─ planos/
│  │  │     └─ login/
│  │  └─ (app)/                  # área do aluno (autenticada)
│  │     ├─ layout.tsx          # sidebar, header
│  │     └─ dashboard/
│  │        ├─ cursos/
│  │        ├─ chat/
│  │        ├─ lives/
│  │        ├─ podcasts/
│  │        ├─ agendamentos/
│  │        ├─ servicos/
│  │        ├─ gestao/
│  │        ├─ propriedades/
│  │        ├─ diagnostico/
│  │        └─ dias-de-campo/
│  │           └─ perfil/
│  └─ (admin)/                  # área administrativa
│     ├─ layout.tsx
│     └─ admin/
│  └─ api/
│     ├─ auth/[...nextauth]/
│     ├─ chat/route.ts
│     ├─ webhooks/gateway/route.ts
│     ├─ youtube/sync/route.ts
│     └─ uploads/route.ts
│  └─ layout.tsx
│  └─ globals.css
├─ components/
│  ├─ ui/                        # shadcn/ui (button, card, dialog...)
│  ├─ layout/                    # sidebar, header, footer
│  ├─ courses/                  # CourseCard, LessonPlayer, ProgressBar
│  ├─ chat/                     # ChatBubble, ChatInput, MessageList
│  └─ shared/                   # EmptyState, LoadingSpinner, ErrorBoundary
├─ lib/
│  ├─ auth.ts                   # config NextAuth
│  ├─ prisma.ts                 # singleton Prisma Client
│  └─ openai.ts                 # cliente OpenAI
```

```

├── youtube.ts      # cliente YouTube Data API
├── gateway.ts      # cliente do gateway de pagto
├── email.ts        # cliente Resend
├── upload.ts       # helpers S3/R2
├── utils.ts        # cn(), formatters
├── constants.ts    # enums, planos, limites
├── server/
│   ├── actions/    # Server Actions por domínio
│   │   ├── courses.ts
│   │   ├── chat.ts
│   │   └── ...
│   ├── services/   # lógica de negócio
│   └── repositories/ # acesso a dados via Prisma
├── hooks/          # React hooks customizados
├── types/          # tipos TypeScript compartilhados
├── middleware.ts   # auth e controle de plano
├── env.ts          # validação de variáveis de ambiente
├── docker-compose.yml # Postgres local
├── .env.example
├── .env.local
├── next.config.js
├── tailwind.config.ts
├── tsconfig.json
└── package.json

```

4.1. Convenções de nomenclatura

- **Pastas e arquivos:** kebab-case (`course-card.tsx` , `dias-de-campo/`).
- **Componentes React:** PascalCase (`CourseCard` , `LessonPlayer`).
- **Hooks:** camelCase com prefixo `use` (`useUser` , `useEnrollment`).
- **Funções e variáveis:** camelCase.
- **Constantes globais:** SCREAMING_SNAKE_CASE.
- **Tipos e interfaces:** PascalCase, sem prefixo `I` .
- **Server Actions:** verbo + substantivo (`createCourse` , `enrollUserInCourse`).

5. Design system

5.1. Identidade visual

O Frutificar tem identidade visual baseada em verde-escuro (campo, natureza, fertilidade), aplicada em fundos escuros para gerar percepção de produto premium e tecnológico.

5.2. Paleta de cores

Cores primárias

Nome	Hex	Uso
 Frutificar Deep	#0e3d1f	Background principal escuro, headers
 Frutificar Forest	#1a5a2e	Background secundário, cards escuros
 Frutificar Green	#2d7a3e	Cor primária de ação (botões, links)
 Frutificar Bright	#4caf50	Hover, acentos, destaques positivos
 Frutificar Light	#a5d6a7	Texto sobre fundo escuro, ícones suaves

Cores neutras

Nome	Hex	Uso
 White	#ffffff	Background light mode, cards claros
 Gray 50	#f5f5f5	Background app light
 Gray 500	#9e9e9e	Texto secundário
 Gray 800	#424242	Texto principal light mode
 Black	#1a1a1a	Background dark mode

Cores de status

Nome	Hex	Uso
 Success	#4caf50	Confirmações, status ótimo
 Warning	#f4a300	Atenção, status crítico
 Danger	#c62828	Erros, status ruim
 Info	#1976d2	Informações neutras

5.3. Tipografia

Fonte principal: **Inter** (via [next/font/google](#)). Alternativa para títulos: **Plus Jakarta Sans**.

Token	Tamanho	Weight	Uso
text-4xl	36px	800	H1 de landing
text-3xl	30px	700	H1 interno
text-2xl	24px	700	H2
text-xl	20px	600	H3, títulos de card
text-base	16px	400	Corpo padrão
text-sm	14px	400	Labels, descrições
text-xs	12px	500	Metadados, tags

5.4. Espaçamento e layout

- Grid base de 4px (Tailwind padrão). Use múltiplos: 4, 8, 12, 16, 24, 32, 48, 64.
- Container máximo de conteúdo: `max-w-7xl` (1280px), com padding lateral `px-4 md:px-6 lg:px-8` .
- Sidebar fixa: 240px desktop, drawer overlay no mobile.
- Border radius padrão: `rounded-lg` (8px) para cards, `rounded-md` (6px) para botões.

5.5. Componentes-chave

- **Button** — variantes: primary (verde), secondary (outline), ghost, destructive. Tamanhos: sm, default, lg.
- **Card** — fundo branco no light, `#1a5a2e` no dark, sombra sutil, padding 24px.
- **Input** — altura 40px, border-radius 6px, focus ring verde.
- **Badge** — para tags de plano, status. Versões: filled, outline, soft.
- **EmptyState** — ilustração + título + descrição + CTA. Usar sempre que listagem estiver vazia.

- **LoadingSpinner** — Skeleton screens para listas, spinner para ações pontuais.

5.6. Tokens CSS (Tailwind config)

```
// tailwind.config.ts
theme: {
  extend: {
    colors: {
      frutificar: {
        deep: '#0e3d1f',
        forest: '#1a5a2e',
        green: '#2d7a3e',
        bright: '#4caf50',
        light: '#a5d6a7',
      }
    },
    fontFamily: {
      sans: ['var(--font-inter)', 'sans-serif'],
      display: ['var(--font-jakarta)', 'sans-serif'],
    }
  }
}
```

6. Estrutura de rotas

6.1. Área pública

- `/` — landing page.
- `/planos` — apresentação dos 3 planos e checkout.
- `/login` e `/cadastro` — autenticação.
- `/recuperar-senha` — recuperação de senha.

6.2. Área do aluno (autenticada)

- `/dashboard` — visão geral.
- `/cursos` e `/cursos/[slug]/[lessonId]` — listagem e player.
- `/chat` — chat com IA.
- `/lives` — agenda e player de lives.
- `/diagnostico` — página informativa com link WhatsApp.
- `/podcasts` — catálogo (Premium e Gold).
- `/agendamentos` — visita técnica (Premium e Gold).
- `/servicos` — solicitações burocráticas (Premium e Gold).
- `/gestao` — downloads (Premium e Gold).
- `/propriedades` — gestão da propriedade rural.
- `/dias-de-campo` — agenda (Gold).
- `/perfil` — dados, assinatura, certificados.

6.3. Área administrativa

- `/admin` — dashboard.
- `/admin/alunos`
- `/admin/cursos`
- `/admin/lives`
- `/admin/agendamentos`
- `/admin/servicos`
- `/admin/podcasts`
- `/admin/downloads`
- `/admin/regras` — regras de recomendação.

6.4. API Routes

- `/api/auth/[...nextauth]`
- `/api/chat` — streaming OpenAI.
- `/api/webhooks/gateway` — eventos de pagamento.
- `/api/youtube/sync` — cron de lives.
- `/api/uploads` — upload S3/R2.

7. Fluxos críticos

7.1. Autenticação e controle de acesso

Middleware central em `src/middleware.ts` intercepta todas as requisições autenticadas e faz, nesta ordem:

1. Verifica sessão válida (NextAuth). Sem sessão, redireciona para `/login`.
2. Para rotas `/admin/*`, verifica role `ADMIN` ou `INSTRUCTOR`.
3. Para rotas premium/gold, consulta subscription do usuário e valida se o plano libera a rota.
4. Se assinatura expirada (`currentPeriodEnd < now`), redireciona para `/perfil/assinatura`.

7.2. Assinatura e ativação

1. Usuário escolhe plano em `/planos`.
2. Server Action chama API do gateway e retorna link de checkout.
3. Usuário paga no checkout do gateway.
4. Gateway dispara webhook para `/api/webhooks/gateway`.
5. Webhook valida assinatura HMAC do payload.
6. Cria ou atualiza `Subscription` com status e `currentPeriodEnd`.
7. Próxima requisição do usuário, middleware libera acesso.

7.3. Chat IA com streaming

1. Usuário envia mensagem no `/chat`.
2. Frontend chama `/api/chat` com a mensagem e ID da sessão.
3. Endpoint monta contexto: prompt base + dados do plano + histórico recente (últimas 10 mensagens).
4. Faz chamada streaming à OpenAI (`gpt-4o-mini` ou `gpt-4o`).
5. Repassa tokens via Server-Sent Events.
6. Frontend renderiza resposta progressivamente.
7. Mensagem completa é persistida no banco em `ChatSession.messages`.

7.4. Sincronização de lives

1. Cron job (Vercel Cron, a cada 5 min) chama `/api/youtube/sync`.
2. Endpoint consulta lives com status `SCHEDULED` ou `LIVE`.
3. Para cada uma, consulta YouTube Data API.

4. Atualiza status no banco.
5. Quando entra em LIVE, dispara e-mail para alunos elegíveis.

7.5. Geração de certificado

1. Aluno marca aula como concluída via Server Action.
2. Service verifica se todas as aulas do curso estão concluídas.
3. Se sim, gera PDF do certificado com nome + curso + data.
4. Salva no S3/R2 e cria registro em `Certificate`.
5. Notifica aluno por e-mail.

8. Boas práticas de desenvolvimento

8.1. Server Components vs Client Components

Regra: Server Component é o padrão. Só use `"use client"` quando precisar de hooks (useState, useEffect), event handlers, ou APIs do browser.

✓ Bom — fetch direto no server

```
// app/(app)/cursos/page.tsx (Server Component)
import { prisma } from '@lib/prisma';
import { CourseCard } from '@components/courses/course-card';

export default async function CoursesPage() {
  const courses = await prisma.course.findMany({ where: { published: true } });
  return courses.map(c => <CourseCard key={c.id} course={c} />);
}
```

✗ Evite — useEffect + fetch quando o dado é estático

```
// Evite: cria waterfall, mais JS no cliente
'use client';
useEffect(() => { fetch('/api/courses').then(...) }, []);
```

8.2. Server Actions para mutações

Use Server Actions para todas as mutações (criar, atualizar, deletar). Valide entrada com Zod, retorne tipo discriminado de sucesso/erro.

```
// src/server/actions/courses.ts
'use server';

import { z } from 'zod';
import { auth } from '@lib/auth';
import { prisma } from '@lib/prisma';

const schema = z.object({
  title: z.string().min(3),
  description: z.string().min(10),
});

export async function createCourse(input: z.infer<typeof schema>) {
  const session = await auth();
  if (session?.user.role !== 'ADMIN') {
    return { ok: false, error: 'Não autorizado' };
  }
  const data = schema.parse(input);
  const course = await prisma.course.create({ data });
  return { ok: true, data: course };
}
```

8.3. Validação de variáveis de ambiente

Use `src/env.ts` com Zod pra validar no boot. Sem isso, erros aparecem só em runtime e em produção.

```
// src/env.ts
import { z } from 'zod';

const envSchema = z.object({
  DATABASE_URL: z.string().url(),
  NEXTAUTH_SECRET: z.string().min(32),
  NEXTAUTH_URL: z.string().url(),
  OPENAI_API_KEY: z.string().startsWith('sk-'),
  YOUTUBE_API_KEY: z.string(),
});

export const env = envSchema.parse(process.env);
```

8.4. Singleton do Prisma Client

Em desenvolvimento, hot reload do Next.js cria múltiplas instâncias do Prisma e estoura conexões. Use o padrão singleton.

```
// src/lib/prisma.ts
import { PrismaClient } from '@prisma/client';

const globalForPrisma = global as unknown as { prisma: PrismaClient };

export const prisma = globalForPrisma.prisma || new PrismaClient();

if (process.env.NODE_ENV !== 'production') globalForPrisma.prisma = prisma;
```

8.5. Tratamento de erros

- Server Actions retornam `{ ok: true, data } | { ok: false, error }`.
- API Routes retornam `NextResponse.json({ error }, { status: 4xx })`.
- Use `error.tsx` e `not-found.tsx` em cada rota crítica.
- Logs estruturados em produção (Pino ou similar).

8.6. Performance

- Use `next/image` para todas as imagens. Configure domínios em `next.config.js`.
- Use `next/font` para fontes — auto-hospedadas e otimizadas.
- Use `loading.tsx` e Suspense para streaming de UI.
- Cache de dados com `revalidateTag` e `revalidatePath`.
- Evite `SELECT *` no Prisma — use `select` explícito.

8.7. Segurança

- Nunca expor `OPENAI_API_KEY`, `DATABASE_URL` ou secrets do gateway no cliente.
- Toda Server Action e API Route verifica sessão e permissão antes de operar.
- Valide TODAS as entradas com Zod, mesmo as que vêm do "seu próprio frontend".
- Webhooks: valide assinatura HMAC do payload.
- Rate limit em endpoints públicos (login, cadastro, chat) — Upstash ou similar.
- Senhas com `bcrypt` (cost factor 12 ou superior).

8.8. Acessibilidade

- Use componentes shadcn/ui — vêm com ARIA correto.
- Labels associadas a inputs.
- Foco visível em todos os elementos interativos.
- Contraste mínimo AA (4.5:1 para texto normal).
- Navegação por teclado funcional em modais e menus.

9. Convenções de código

9.1. Imports

Ordem dos imports:

1. React e Next.js
2. Bibliotecas externas
3. Aliases internos (`@/lib` , `@/components` , `@/server`)
4. Imports relativos
5. Types

Use aliases via `@/*` configurados em `tsconfig.json` . Nunca `../../../../` .

9.2. Componentes

- Um componente por arquivo.
- Props tipadas com interface ou type.
- Componentes pequenos e focados (regra de ouro: se passou de 200 linhas, quebra).
- Composição em vez de props condicionais excessivas.

9.3. Git

- Commits em português, no imperativo: `adiciona player de aula` , `corrige bug do progresso` .
- Branches: `feat/nome-da-feature` , `fix/nome-do-bug` , `chore/tarefa` .
- Commits pequenos e atômicos — facilita review e rollback.

9.4. TypeScript

- `strict: true` no `tsconfig.json` .
- Nunca use `any` . Se realmente precisar, use `unknown` e faça type guard.
- Prefira `type` a `interface` , exceto quando for estender classes.
- Tipos de retorno explícitos em funções públicas e Server Actions.

9.5. Estilização

- Tailwind only — não criar CSS Modules nem styled-components.
- Use `cn()` (clsx + tailwind-merge) para classes condicionais.

- Classes longas: quebre em variáveis ou componentes.
- Dark mode via `dark:` prefix do Tailwind.

10. Checklist por sprint

Sprint 1 — Fundação (dias 1 a 7)

- ☐ Projeto Next.js criado com TypeScript, Tailwind, App Router
- ☐ Prisma configurado e conectando ao Postgres local (Docker)
- ☐ Schema completo do banco com todas as entidades
- ☐ Migrations iniciais rodando
- ☐ Seeds com planos, admin e curso de exemplo
- ☐ NextAuth configurado com login por email/senha
- ☐ Middleware de autenticação funcionando
- ☐ shadcn/ui instalado com componentes base
- ☐ Layout do dashboard (sidebar + header) com identidade visual
- ☐ Landing page e página de planos (estáticas)
- ☐ Páginas de login, cadastro e recuperação de senha

Sprint 2 — Cursos (dias 8 a 14)

- ☐ CRUD admin de cursos, módulos e aulas
- ☐ Player de aula com YouTube embed
- ☐ Marcação de progresso por aula
- ☐ Upload e download de materiais PDF
- ☐ Listagem de cursos filtrada por plano
- ☐ Geração automática de certificado em PDF
- ☐ Página "meus certificados" no perfil

Sprint 3 — Features Premium e integrações (dias 15 a 21)

- ☐ Chat IA com OpenAI streaming
- ☐ Histórico de conversa persistido
- ☐ Página de diagnóstico com link WhatsApp
- ☐ Sistema de agendamento de visita técnica
- ☐ Sistema de solicitação de serviços
- ☐ Catálogo de podcasts
- ☐ Integração YouTube Data API + cron de sync

- ☐ Página de lives com player
- ☐ Agenda de dias de campo
- ☐ Menu de gestão (downloads)

Sprint 4 — Pagamento e entrega (dias 22 a 30)

- ☐ Integração com gateway de pagamento
- ☐ Webhook de assinatura funcionando
- ☐ Middleware de controle de plano completo
- ☐ Módulo de gestão de propriedade rural
- ☐ E-mails transacionais (boas-vindas, confirmação)
- ☐ Tela de gestão de assinatura no perfil
- ☐ Revisão visual e ajustes finais
- ☐ Testes de fluxo ponta a ponta
- ☐ Deploy em produção
- ☐ Treinamento do cliente